

Seguridad informática

TP2 – Criptografia

Integrantes:

- Carrano Paula
- Colón Lucrecia
- Lovato Lautaro
- Menager Emanuel

Fecha: 27/06/2026

Objetivo

El objetivo del programa es cifrar y descifrar archivos usando AES-256. El usuario puede tomar un archivo original, generar una versión cifrada con extensión .enc y luego recuperar el archivo original usando la misma contraseña.

El programa fue desarrollado en Python y utiliza la librería **cryptography**, que provee implementaciones seguras de algoritmos criptográficos.

Funcionamiento general

El archivo principal del proyecto es **aes_file_tool.py**. Este programa funciona desde la consola y recibe tres argumentos:

```
python aes_file_tool.py accion archivo_entrada archivo_salida
```

La acción puede ser:

- **encrypt**: cifrar el archivo de entrada.
- **decrypt**: descifra el archivo de entrada.

La contraseña se solicita con getpass, por lo que no se muestra en pantalla mientras se escribe.

Proceso de cifrado

Cuando se ejecuta la opción **encrypt**, el programa realiza los siguientes pasos:

1. Lee el archivo original como bytes.
2. Solicita una contraseña al usuario.
3. Genera un salt aleatorio de 16 bytes.
4. Genera un nonce aleatorio de 12 bytes.
5. Deriva una clave de 256 bits a partir de la contraseña usando PBKDF2-HMAC-SHA256.
6. Cifra el contenido usando AES-256-GCM.
7. Guarda en el archivo de salida el encabezado, el salt, el nonce y el contenido cifrado.

El salt permite que la misma contraseña genere claves diferentes en distintos cifrados. El nonce es necesario para el modo AES-GCM y también se genera de forma aleatoria para cada archivo cifrado.

Proceso de descifrado

Cuando se ejecuta la opción **decrypt**, el programa realiza los siguientes pasos:

1. Lee el archivo cifrado .enc.
2. Valida que el archivo tenga el encabezado esperado.
3. Recupera el salt y el nonce guardados durante el cifrado.
4. Solicita al usuario la misma contraseña usada para cifrar.
5. Deriva nuevamente la clave de 256 bits usando PBKDF2-HMAC-SHA256.
6. Descifra el contenido usando AES-256-GCM.
7. Guarda el archivo original recuperado en la ruta indicada.

Si la contraseña es incorrecta o el archivo cifrado fue modificado, el descifrado falla y el programa muestra un mensaje de error.

Elección de AES-GCM

Se utiliza AES-256-GCM porque permite cifrar el contenido y, al mismo tiempo, verificar su integridad. Esto significa que el programa no solo protege la confidencialidad del archivo, sino que también puede detectar si el archivo fue alterado o si se intentó descifrar con una contraseña incorrecta.

Pruebas automatizadas

El proyecto incluye pruebas automatizadas en **test_aes_file_tool.py**. Estas pruebas verifican los siguientes casos:

- Cifrado y descifrado correcto de texto.
- Cifrado y descifrado correcto de datos binarios.
- Cifrado y descifrado de un archivo vacío.
- Falla al descifrar con una contraseña incorrecta.
- Falla al descifrar un archivo cifrado alterado.
- Falla al intentar descifrar un archivo con formato invalido.

Para ejecutar las pruebas:

```
pip install -r requirements-dev.txt  
python -m pytest
```

ANEXO: Instrucciones de ejecución

1. Instalar dependencias

Desde la carpeta del proyecto, ejecutar:

pip install -r requirements.txt

2. Cifrar un archivo

El proyecto incluye una carpeta de ejemplo con archivos para probar:

ejemplos/
 para_encryptar/
 encriptados/
 desencriptados/

Para cifrar el archivo de ejemplo:

**python aes_file_tool.py encrypt ejemplos/para_encryptar/mensaje.txt
ejemplos/encriptados/mensaje.txt.enc**

El programa pedirá una contraseña:

Contraseña:

Al escribirla, no se verá ningún carácter en pantalla. Esto es normal y sirve para proteger la contraseña.

El archivo cifrado se genera en:

ejemplos/encriptados/mensaje.txt.enc

3. Descifrar un archivo

Para recuperar el archivo original:

**python aes_file_tool.py decrypt ejemplos/encriptados/mensaje.txt.enc
ejemplos/desencriptados/mensaje_recuperado.txt**

Se debe ingresar la misma contraseña usada durante el cifrado.

El archivo recuperado se generará en:

ejemplos/desencriptados/mensaje_recuperado.txt

4. Ejecutar pruebas

Para instalar las dependencias de desarrollo:

```
pip install -r requirements-dev.txt
```

Luego ejecutar:

```
python -m pytest
```